

Three Frobenius proofs, read slowly

Lean 4 $\longleftrightarrow$ mathematics $\longleftrightarrow$ type theory

Abstract

Three small proofs. Char 2.

- **frob_bijective**: $x \mapsto x^{2^k}$ is a bijection.
- **frob_add**: $(x + y)^{2^k} = x^{2^k} + y^{2^k}$.
- **sq_add**: $(x + y)^2 = x^2 + y^2$.

We read each line as: *math* $\leftrightarrow$ *Lean code* $\leftrightarrow$ *a typing judgement* $\Gamma \vdash e : \tau$.

Setting

Fix the context Γ once. It is the list of assumptions in scope.

$$\Gamma = (F : \text{Type}, [\text{Field } F], [\text{Fintype } F], [\text{CharP } F \ 2])$$

- F is a **field**.
- F is **finite** ($\text{Fintype} \Rightarrow \text{Finite}$).
- F has **characteristic 2**: $1 + 1 = 0$, so $x + x = 0$.

In Lean, $[\dots]$ are *instance* arguments. They are found automatically by type-class search. So Γ is really “ F plus three instances the elaborator carries around for free”.

1 **frob_bijective** : the map $x \mapsto x^{2^k}$ is a bijection

Statement

$$\forall k : \mathbb{N}. \text{Bijjective } (\lambda x:F. x^{2^k})$$

```

lemma frob_bijective (k : ℕ) :
  Function.Bijective (fun x : F => x ^ (2 ^ k)) := by
  have : (fun x : F => x ^ (2 ^ k)) = iterateFrobenius F 2 k := by
    ext x; simp [iterateFrobenius_def]
  rw [this]
  exact Finite.injective_iff_bijective.mp (iterateFrobenius F 2 k).injective

```

As a typing goal, the proof term we must build is:

$$\Gamma, k : \mathbb{N} \vdash ? : \text{Bijective } (\lambda x. x^{2^k})$$

The plan

The bare lambda $\lambda x. x^{2^k}$ is *the same map* as a bundled object Mathlib already knows: the iterated Frobenius φ^k where $\varphi(x) = x^2$.

$$\underbrace{\lambda x. x^{2^k}}_{\text{raw function}} = \underbrace{\text{iterateFrobenius } F \ 2 \ k}_{\text{a bundled ring hom } F \rightarrow_{+*} F}$$

Once we know the raw map *is* a ring hom on a *finite* field, two facts finish it:

1. a ring hom out of a field is **injective**;
2. on a **finite** type, injective $\Rightarrow$ bijective (pigeonhole).

Step 1 — replace the raw map by the bundled one

```

have : (fun x : F => x ^ (2 ^ k)) = iterateFrobenius F 2 k := by
  ext x; simp [iterateFrobenius_def]

```

- `have : ... := ...` introduces a local proof of an equality *of functions*.
- `ext x` is **function extensionality**: to prove $f = g$ it suffices to prove $\forall x, f x = g x$.

$$\frac{\forall x. f x = g x}{f = g} \text{ (FUNEXT)}$$

- `simp [iterateFrobenius_def]` closes $x^{2^k} = (\text{iterateFrobenius } F \ 2 \ k) x$ because the rewrite rule

$$\text{iterateFrobenius_def} : (\text{iterateFrobenius } R \ p \ n) x = x^{p^n}$$

turns the right side into x^{2^k} , leaving $x^{2^k} = x^{2^k}$, i.e. `rfl`.

What is `iterateFrobenius F 2 k`? It is the k -fold composite of the Frobenius endomorphism $\varphi : x \mapsto x^2$, packaged as a term of type $F \rightarrow^{++} F$ (a *ring homomorphism*). The arrow $\uparrow / \uparrow$ silently turns this bundle back into a plain function when needed.

Step 2 — rewrite the goal

```
rw [this]
```

`rw` substitutes `left` $\rightarrow$ `right` using the equation `this`. The goal mutates:

$$\text{Bijjective } (\lambda x. x^{2^k}) \quad \xrightarrow{\text{rw [this]}} \quad \text{Bijjective } (\uparrow(\text{iterateFrobenius } F \ 2 \ k))$$

Step 3 — injective, then bijective

```
exact Finite.injective_iff_bijective.mp (iterateFrobenius F 2 k).injective
```

Read it right to left.

$$\underbrace{(\text{iterateFrobenius } F \ 2 \ k).injective}_{: \text{Injective } \varphi^k} \quad \xrightarrow{\cdot \text{mp}} \quad \underbrace{\text{Bijjective } \varphi^k}_{\text{goal}}$$

- `(...).injective` : a ring hom on a field is injective (its kernel is an ideal of a field, hence $\{0\}$).

$$\Gamma \vdash \varphi^k.\text{injective} : \text{Injective } (\uparrow\varphi^k)$$

- `Finite.injective_iff_bijective` : for a self-map of a *finite* type,

$$\text{Injective } f \leftrightarrow \text{Bijjective } f.$$

This is the pigeonhole principle:

$$\frac{[\text{Finite } F] \quad f : F \rightarrow F \quad \text{Injective } f}{\text{Surjective } f}$$

and $\text{injective} \wedge \text{surjective} = \text{bijective}$.

- `.mp` takes the forward direction of that $\leftrightarrow$.
- `exact` says: *this term has exactly the goal's type*. Done.

The whole derivation as one tree

$$\frac{\frac{[\text{Finite } F]}{\text{Injective } \varphi^k \leftrightarrow \text{Bijective } \varphi^k} \quad \frac{\varphi^k : F \rightarrow_{+*} F}{\text{Injective } \varphi^k} \text{FINITE} \quad \text{FIELD}}{\text{Bijective } \varphi^k} \text{MP}$$

Picture

$$\begin{array}{ccc} F & \xrightarrow{\lambda x. x^{2^k}} & F \\ \parallel & & \parallel \\ F & \xrightarrow[\uparrow \varphi^k]{\sim} & F \end{array} \quad \begin{array}{ccc} \text{inj} & \xrightarrow[\wedge]{\text{finite}} & \text{surj} \\ & & \text{bij} \end{array}$$

Top row = raw map. Bottom row = same map, bundled, and an isomorphism ($\sim$). The vertical equalities are Step 1. ■

2 `frob_add` : the Freshman's Dream, iterated

Statement

Now the context is only a **commutative ring** R of char 2 (no finiteness needed).

$$\Gamma' = (R : \text{Type}, [\text{CommRing } R], [\text{CharP } R \ 2])$$

$$\forall k : \mathbb{N}. \forall x, y : R. \quad (x + y)^{2^k} = x^{2^k} + y^{2^k}$$

```

lemma frob_add (k : ℕ) (x y : R) :
  (x + y) ^ (2 ^ k) = x ^ (2 ^ k) + y ^ (2 ^ k) :=
  add_pow_char_pow_of_commute 2 k (Commute.all x y)

```

This is a term proof — no by

There is no tactic block. The body *is* the proof term. We just **apply a library lemma** to the right arguments. The skill is matching its type to ours.

The library lemma:

$$\text{add_pow_char_pow_of_commute} : \quad \Pi (p : \mathbb{N}) [\text{Fact } p.\text{Prime}] [\text{CharP } R p] (n : \mathbb{N}),$$

$$\text{Commute } x \ y \longrightarrow (x + y)^{p^n} = x^{p^n} + y^{p^n}$$

We feed it $p := 2$ and $n := k$:

slot	we supply
p	$\mapsto 2$
$[\text{Fact } p.\text{Prime}]$	$\mapsto$ found by instance search (2 is prime)
$[\text{CharP } R 2]$	$\mapsto$ in Γ'
n	$\mapsto k$
$\text{Commute } x \ y$	$\mapsto \text{Commute.all } x \ y$

`Commute.all x y` : in a *commutative* ring, every pair commutes, so $x \cdot y = y \cdot x$ is free. The general lemma only needs commutativity of the two base elements — here that is automatic.

Why the theorem is true (the math under the name)

Binomial expansion:

$$(x + y)^{2^k} = \sum_{i=0}^{2^k} \binom{2^k}{i} x^i y^{2^k-i}.$$

In char 2 every interior coefficient vanishes:

$$\binom{2^k}{i} \equiv 0 \pmod{2} \quad (0 < i < 2^k),$$

by Kummer / Lucas. Only the two endpoints survive:

$$(x + y)^{2^k} = x^{2^k} + y^{2^k}. \quad (*)$$

The map $\varphi : x \mapsto x^2$ is therefore **additive**; iterating k times gives $(*)$. Mathlib has bottled exactly this as the lemma above.

Typing judgement

$\Gamma', k : \mathbb{N}, x : R, y : R \vdash (\text{add_pow_char_pow_of_commute } 2 \ k \ (\text{Commute.all } x \ y)) : (x + y)^{2^k} = x^{2^k} + y^{2^k}$

The whole proof is one application node:

$$\frac{\text{add_pow_char_pow_of_commute } 2 \ k : \text{Commute } x \ y \rightarrow (x + y)^{2^k} = x^{2^k} + y^{2^k} \quad \text{Commute.all } x \ y : \text{Commute } x \ y}{(x + y)^{2^k} = x^{2^k} + y^{2^k}} \text{ APP}$$

■

3 sq_add : one line, by specialisation

Statement

$$\forall x, y : R. \quad (x + y)^2 = x^2 + y^2$$

```
lemma sq_add (x y : R) : (x + y) ^ 2 = x ^ 2 + y ^ 2 := frob_add 1 x y
```

It is just **frob_add** at $k = 1$

$$\text{frob_add } 1 \ x \ y : (x + y)^{2^1} = x^{2^1} + y^{2^1}$$

The expected type wants the exponent 2, the term gives 2^1 . These are accepted as the same goal because they are **definitionally equal**:

$$2^1 \equiv 2 \cdot 2^0 \equiv 2 \cdot 1 \equiv 2.$$

Definitional equality ($\equiv$) is the silent equality the kernel checks by *computation*, with no proof step. Here 2^1 *reduces* to 2, so Lean sees **frob_add 1 x y** as already having the desired type. No **rw**, no **simp** — just reduction.

Typing judgement

$$\Gamma', x : R, y : R \vdash \text{frob_add } 1 \ x \ y : \underbrace{(x + y)^{2^1} = x^{2^1} + y^{2^1}}_{\equiv (x+y)^2 = x^2 + y^2}$$

$$\frac{\text{frob_add } 1 \ x \ y : (x + y)^{2^1} = x^{2^1} + y^{2^1}}{(x + y)^2 = x^2 + y^2} \text{ CONV } (2^1 \equiv 2)$$

Dependency picture

$$\text{add_pow_char_pow_of_commute} \xrightarrow{p:=2, n:=k} \text{frob_add} \xrightarrow{k:=1} \text{sq_add}$$

Each arrow is “substitute, and let the kernel reduce”. The three lemmas form a tiny tower: a Mathlib root, one named iteration, one specialisation. ■

One-screen summary

lemma	idea	engine
<code>frob_bijective</code>	$x \mapsto x^{2^k}$ is 1–1 and onto	inj. (field) + finite $\Rightarrow$ bij.
<code>frob_add</code>	$(x + y)^{2^k} = x^{2^k} + y^{2^k}$	Freshman’s Dream, char 2
<code>sq_add</code>	$(x + y)^2 = x^2 + y^2$	<code>frob_add</code> at $k = 1$, $2^1 \equiv 2$

Three flavours of proof, one per lemma:

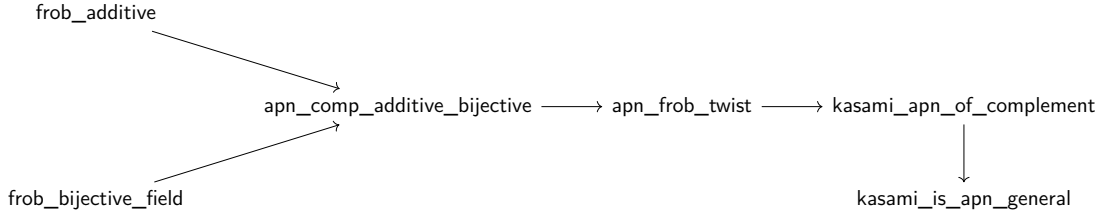
$$\underbrace{\text{frob_bijective}}_{\text{tactic: have / rw / exact}} \quad \underbrace{\text{frob_add}}_{\text{term: apply a library lemma}} \quad \underbrace{\text{sq_add}}_{\text{term: specialise + reduce}}$$

Follow-up note: one thread, bottom to top

The two char-2 facts above are not curiosities. They are the **two hypotheses** of a single propagation lemma, and that lemma is exactly the lever that lifts the Kasami APN theorem from *odd k only* to *all k*.

This note follows that one thread, from the two tiny lemmas up to the headline result. Every step is real code from `RequestProject/Kasami/Classical/KasamiEvenK.lean`.

The thread, named:



Left: the two char-2 inputs. Middle: a generic preservation principle and its Frobenius specialisation. Right: the Kasami exponent congruence feeds that principle, and the case split finishes.

4 The hinge: APN is preserved by an additive bijection

First recall what **APN** means in this project (file `KasamiAPN.lean`):

```

def IsAPN {F : Type*} [Field F] [CharP F 2] (f : F → F) : Prop := ∀
  (a : F), a ≠ 0 → ∀ (x y : F),
    f (x + a) + f x = f (y + a) + f y → y = x ∨ y = x + a

```

In char 2, $u-v = u+v$, so a “differential collision” is just the equation $f(x+a)+f(x) = f(y+a)+f(y)$. **APN** = “every collision is trivial”: y must be one of the only two forced solutions x or $x+a$.

The hinge lemma says: **post-composing an APN map with an additive bijection σ stays APN.**

```

lemma apn_comp_additive_bijective {F : Type*} [Field F] [CharP F 2]
  {f : F → F} (hf : IsAPN f) σ
  { : F → F} (σh_bij : Function.Bijective σ)
  (σh_add : ∀ x y, σ(x + y) = σx + σy) :
  IsAPN σ( of ) := by
  intro a ha x y hxy
  convert hf a ha x y _ using 1
  exact σh_bij.injective ( by aesop )

```


Why this is the right abstraction

Look at a collision for $\sigma \circ f$ and push σ through:

$$\sigma(f(x+a)) + \sigma(f(x)) \stackrel{h\sigma_{add}}{=} \sigma(f(x+a) + f(x)).$$

So a collision of $\sigma \circ f$ is σ applied to a collision of f :

$$\sigma(f(x+a) + f(x)) = \sigma(f(y+a) + f(y)).$$

Now σ **injective** cancels:

$$f(x+a) + f(x) = f(y+a) + f(y),$$

and f being APN delivers $y = x \vee y = x + a$ — which is exactly the goal, since $\sigma \circ f$ has the same two forced solutions.

The proof body is three lines, and each maps to one phrase above. `intro a ha x y hxy` unpacks the collision hypothesis `hxy` for $\sigma \circ f$. `convert hf a ha x y _ using 1` reduces the goal to the remaining premise: that f also collides at x, y . `hσ_bij.injective` (by `aesop`) discharges it — `aesop` rewrites the $\sigma \circ f$ collision through `hσ_add` into $\sigma(\dots) = \sigma(\dots)$, and `.injective` strips the outer σ .

As an inference rule

$$\frac{\text{IsAPN } f \quad \text{Bijective } \sigma \quad \forall x y. \sigma(x+y) = \sigma x + \sigma y}{\text{IsAPN } (\sigma \circ f)} \text{ APN-TRANSPORT}$$

The two side premises are precisely the shapes of `frob_bijective` and `frob_add`. That is the whole point of this note.

5 Plugging in Frobenius: the twist lemma

Now take $\sigma = \text{Frobenius}$. In this file the two inputs reappear as:

```
lemma frob_additive {F : Type*} [CommSemiring F] [CharP F 2]
  (j : ℕ) (x y : F) :
  (x + y) ^ (2 ^ j) = x ^ (2 ^ j) + y ^ (2 ^ j) :=
  add_pow_char_pow (p := 2) (n := j) x y

lemma frob_bijective_field {F : Type*} [Field F] [Fintype F] [CharP F 2]
  (j : ℕ) : Function.Bijective (fun x : F => x ^ (2 ^ j)) := by
  have : (fun x : F => x ^ (2 ^ j)) = iterateFrobenius F 2 j := by
```

```

    ext x; simp [iterateFrobenius_def]
  rw [this]
  exact Finite.injective_iff_bijective.mp (iterateFrobenius F 2 j).injective

```

These are literally `frob_add` and `frob_bijective` from Part I, re-stated in this file under the names the Kasami development uses. `frob_additive` is the additivity premise; `frob_bijective_field` is the bijectivity premise.

Feed both into the hinge and you get: **multiplying an APN exponent by a power of 2 keeps it APN.**

```

lemma apn_frob_twist {F : Type*} [Field F] [Fintype F] [CharP F 2]
  {d : ℕ} (hd : IsAPN (fun x : F => x ^ d))
  (j : ℕ) :
  IsAPN (fun x : F => x ^ (d * 2 ^ j)) := by
have heq : (fun x : F => x ^ (d * 2 ^ j)) =
  (fun x => x ^ (2 ^ j)) ∘ (fun x => x ^ d) := by
  ext x; simp [pow_mul]
rw [heq]
exact apn_comp_additive_bijective hd (frob_bijective_field j) (frob_additive j)

```

The one line that matters

The last line is the entire thread compressed:

$$\underbrace{\text{apn_comp_additive_bijective}}_{\text{APN-TRANSPORT}} \underbrace{\text{hd}}_{\text{IsAPN}(x^d)} \underbrace{(\text{frob_bijective_field } j)}_{\text{Bijective}(x^{2^j})} \underbrace{(\text{frob_additive } j)}_{\sigma(x+y)=\sigma x+\sigma y}.$$

The rewrite `heq` uses $x^{d \cdot 2^j} = (x^{2^j}) \circ (x^d)$, i.e. `pow_mul`, to expose the composite $\sigma \circ f$ with $f = x^d$ and $\sigma = x^{2^j}$.

$$\frac{\text{IsAPN}(x^d) \quad \text{Bijective}(x^{2^j}) \text{ (FROB)} \quad (x+y)^{2^j} = x^{2^j} + y^{2^j} \text{ (FROB)}}{\text{IsAPN}(x^{d \cdot 2^j})} \text{ APN-TRANSPORT}$$

6 The arithmetic: d_k is a Frobenius twist of d_{n-k}

The Kasami exponent is $d_k = 2^{2k} - 2^k + 1$:

```
def kasamiExp (k : ℕ) : ℕ := 2 ^ (2 * k) - 2 ^ k + 1
```

The key number fact: modulo $2^n - 1$, the exponent d_k equals d_{n-k} *twisted* by 2^{2k} .

```
lemma kasami_exp_congr_mod {k n : ℕ} (hk : 0 < k) (hkn : k < n) :
  kasamiExp k % (2 ^ n - 1) =
  (kasamiExp (n - k) * 2 ^ (2 * k)) % (2 ^ n - 1) := by ...
```

$$d_k \equiv d_{n-k} \cdot 2^{2k} \pmod{2^n - 1}.$$

Why mod $2^n - 1$? On $\text{GF}(2^n)^\times$ every element has order dividing $2^n - 1$, so x^e depends only on $e \bmod (2^n - 1)$. A congruence of exponents becomes an *equality of power maps*.

Transported to the field, the congruence becomes a pointwise identity (the $x = 0$ case is separate, the $x \neq 0$ case uses $x^{2^n} = x$ and the congruence):

```
lemma kasami_pow_frob_identity {F : Type*} [Field F] [Fintype F] [CharP F 2]
  {n : ℕ} (hn : Fintype.card F = 2 ^ n)
  (k : ℕ) (hk : 0 < k) (hkn : k < n) (x : F) :
  x ^ kasamiExp k = (x ^ kasamiExp (n - k)) ^ (2 ^ (2 * k)) := by ...
```

$$x^{d_k} = (x^{d_{n-k}})^{2^{2k}} = \sigma(x^{d_{n-k}}), \quad \sigma = \text{Frob}_{2^k}.$$

7 Closing the loop: APN of the complement

The two halves — “ d_k is d_{n-k} twisted” and “twisting preserves APN” — snap together:

```
lemma kasami_apn_of_complement {F : Type*} [Field F] [Fintype F] [CharP F 2]
  {n : ℕ} (hn : Fintype.card F = 2 ^ n)
  (k : ℕ) (hk : 0 < k) (hkn : k < n)
  (hpn : IsAPN (fun x : F => x ^ kasamiExp (n - k))) :
  IsAPN (fun x : F => x ^ kasamiExp k) := by
  have heq : (fun x : F => x ^ kasamiExp k) =
```

```

    (fun x : F => x ^ (kasamiExp (n - k) * 2 ^ (2 * k))) := by
    ext x; rw [pow_mul]; exact kasami_pow_frob_identity hn k hk hkn x
  rw [heq]
  exact apn_frob_twist hpn (2 * k)

```

Read the last two lines as the thread closing: `heq` rewrites x^{d_k} into $x^{d_{n-k} \cdot 2^{2k}}$ (the arithmetic of §3), then `apn_frob_twist hpn (2*k)` applies the twist lemma of §2 — which is the hinge of §1 fed with the two char-2 lemmas.

$$\frac{\text{IsAPN}(x^{d_{n-k}}) \quad x^{d_k} = (x^{d_{n-k}})^{2^{2k}} \text{ (§3)}}{\text{IsAPN}(x^{d_k})} \text{ COMPLEMENT}$$

8 The headline: the parity restriction disappears

The *original* Kasami theorem (`KasamiAPN.lean`) carries `(hk_odd : Odd k)`:

```

theorem kasami_is_apn {F : Type*} [Field F] [Fintype F] [CharP F 2]
  {n : ℕ} (hn : Fintype.card F = 2 ^ n) (k : ℕ)
  (hk : 1 < k) (hk_odd : Odd k) (hkn : k < n)
  (hn_odd : Odd n) (hcop : Nat.Coprime k n) :
  IsAPN (fun (x : F) => x ^ (kasamiExp k)) := ...

```

When k is **even** and n is **odd**, the complement $n - k$ is **odd** with the *same* coprimality — so `kasami_is_apn` applies to $n - k$, and `kasami_apn_of_complement` carries it back to k :

```

theorem kasami_is_apn_even_k {F : Type*} [Field F] [Fintype F] [CharP F 2]
  {n : ℕ} (hn : Fintype.card F = 2 ^ n) (k : ℕ)
  (hk : 1 < k) (hk_even : Even k) (hkn : k < n)
  (hn_odd : Odd n) (hcop : Nat.Coprime k n)
  (hnk : n - k ≥ 2) :
  IsAPN (fun (x : F) => x ^ kasamiExp k) := by
  apply kasami_apn_of_complement hn k (by omega) hkn
  have hcop' : Nat.Coprime (n - k) n := coprime_sub_self (le_of_lt hkn) hcop
  have hodd' : Odd (n - k) := odd_sub_even hn_odd hk_even (le_of_lt hkn)
  exact kasami_is_apn hn (n - k) hnk hodd' (by omega) hn_odd hcop'

```

The edge case $n-k = 1$ is handled separately (`kasami_is_apn_even_k_edge`): there the complement exponent is the Gold exponent $d_1 = 3 = 2^1 + 1$, and Gold APN is proved directly (`gold_is_apn`). Same complement trick, different base case.

The unified statement — **no parity hypothesis on k** :

```

theorem kasami_is_apn_general {F : Type*} [Field F] [Fintype F] [CharP F 2]
  {n : ℕ} (hn : Fintype.card F = 2 ^ n) (k : ℕ)
  (hk : 1 < k) (hkn : k < n)
  (hn_odd : Odd n) (hcop : Nat.Coprime k n) :
  IsAPN (fun (x : F) => x ^ kasamiExp k) := by
  by_cases hk_odd : Odd k
  · exact kasami_is_apn hn k hk hk_odd hkn hn_odd hcop
  · have hk_even : Even k := Nat.not_odd_iff_even.mp hk_odd
    by_cases hnk : n - k ≥ 2
    · exact kasami_is_apn_even_k hn k hk hk_even hkn hn_odd hcop hnk
    · have hnk1 : n - k = 1 := by omega
      exact kasami_is_apn_even_k_edge hn k hk hk_even hkn hn_odd hcop hnk1

```

Compare the two signatures: `kasami_is_apn` has `(hk_odd : Odd k)`; `kasami_is_apn_general` does *not*. That single deleted hypothesis is what the whole thread bought.

The thread, on one screen

step	what it adds
<code>frob_additive</code>	σ is additive (char 2)
<code>frob_bijective_field</code>	σ is a bijection (field + finite)
<code>apn_comp_additive_bijective</code>	inj. + additive $\Rightarrow$ APN survives $\sigma \circ (-)$
<code>apn_frob_twist</code>	specialise $\sigma = x^{2^j}$: x^d APN $\Rightarrow x^{d \cdot 2^j}$ APN
<code>kasami_exp_congr_mod</code>	$d_k \equiv d_{n-k} \cdot 2^{2k} \pmod{2^n - 1}$
<code>kasami_pow_frob_identity</code>	$x^{d_k} = (x^{d_{n-k}})^{2^{2k}}$ on the field
<code>kasami_apn_of_complement</code>	APN at $n - k \Rightarrow$ APN at k
<code>kasami_is_apn_general</code>	drop Odd k

One sentence: *Frobenius is an additive bijection (`frob_add + frob_bijective`), an additive bijection preserves APN, the Kasami exponent d_k is a Frobenius twist of its complement d_{n-k} , and so the odd- k theorem covers the even- k case by swapping $k \leftrightarrow n - k$ — erasing the parity restriction.*

<code>frob_add + frob_bijective</code> $\implies$ APN preserved by twist $\implies$ <code>kasami_is_apn_general</code>
